KONGU ENGINEERING COLLEGE

INDUSTRIAL DG MONITORING AND ANALYTICS SYSTEM USING ESP32 AND IOT

MECHATRONICS AND IOT

EXCERSICE – 13

HEMAVARSHINI S - 24MER017

KAVIN KUMARAN S - 24MER023

KOWSIKA K - 24MER026

VISHNUPRASATH B - 24MER062

INDUSTRIAL DG MONITORING AND ANALYTICS SYSTEM USING ESP32 AND IOT

1. Project Overview

Introduction

Industrial diesel generators (DG sets) are widely used as backup and continuous power sources, and their reliable operation depends on continuous awareness of key electrical and environmental parameters. Manual, periodic inspection of a DG set does not provide continuous visibility into its operating condition and can delay the detection of abnormal parameters.

The Industrial DG Monitoring and Analytics System is an IoT-based system built around an ESP32 microcontroller that continuously monitors the operating conditions of a diesel generator. The system measures voltage, current, temperature, and humidity using sensors connected to the ESP32, and calculates the approximate electrical power drawn by the DG set.

The measured parameters are displayed locally on an OLED display for immediate on-site viewing and are also transmitted through Wi-Fi to the ThingSpeak IoT cloud platform, where the data can be visualized as graphs and monitored remotely. This combination of local display and cloud-based analytics supports both immediate observation and longer-term performance analysis of the DG set.

The main objective is to provide continuous, automated DG monitoring with both local and remote visibility, supporting early identification of abnormal operating conditions and better maintenance planning.

Project Objectives

- To monitor DG electrical parameters in real time.
- To measure generator voltage and current.
- To calculate the approximate electrical power.
- To monitor temperature and humidity around the DG.
- To display the measured values locally using an OLED.
- To transmit the data to ThingSpeak using Wi-Fi.
- To provide remote monitoring and basic analytics.
- To help identify abnormal operating conditions early.

2. Problem Statement

Industrial DG sets are critical backup and continuous power sources for manufacturing plants, commercial buildings, hospitals, and data centers. Conventional DG monitoring commonly relies on manual, periodic inspection, which does not provide continuous or remote visibility into the generator's actual operating condition.

This can result in:

- Delayed detection of abnormal voltage or current conditions.
- Unnoticed temperature or humidity changes around the DG set.
- Difficulty tracking historical performance trends for maintenance planning.
- Lack of remote visibility for supervisors and maintenance teams.
- Increased risk of unplanned downtime due to undetected issues.
- Inefficient, purely time-based maintenance scheduling.

Therefore, a system is required to continuously monitor key DG parameters, display them locally, and transmit them to a cloud platform for remote monitoring and basic performance analytics.

3. Proposed Solution

The proposed system uses multiple sensors and a microcontroller to automate DG monitoring and analytics. The voltage sensor measures the generator's output voltage, the ACS712 current sensor measures the current drawn by the DG system, and the DHT11 sensor monitors the surrounding temperature and humidity. All sensor outputs are processed by the ESP32.

The ESP32 calculates the electrical power using the relationship:

Power = Voltage × Current

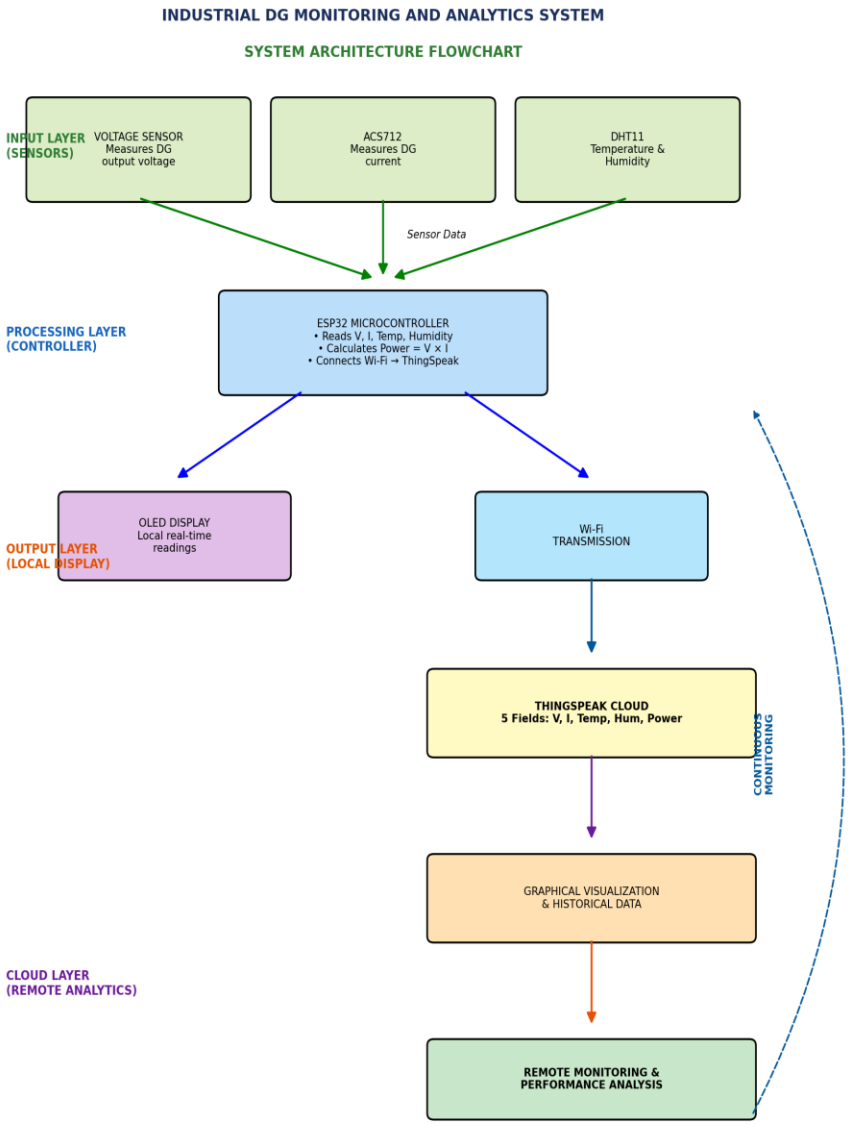
The measured parameters are shown on the OLED display for local monitoring, and the ESP32 uses its built-in Wi-Fi capability to send the readings to ThingSpeak:

Voltage Sensor + ACS712 + DHT11 → ESP32 → Power Calculation → OLED Display + ThingSpeak

This enables both immediate local observation of DG conditions and remote, graph-based monitoring and analytics through the ThingSpeak cloud dashboard.

4. System Architecture

The overall architecture of the system is:



System Flow:

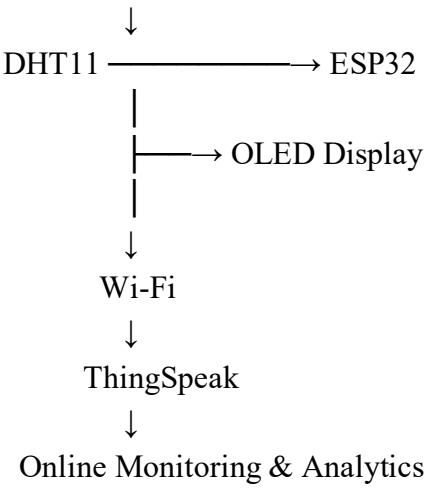
DG SET

↓

Voltage Sensor ─┐

|

ACS712 ─┤



5. Components Used

S.No	Component	Quantity	Purpose
1	ESP32 MCU	1	Main controller and Wi-Fi communication
2	ACS712 Current Sensor	1	Measures DG current
3	Voltage Sensor Module	1	Measures voltage
4	DHT11 Sensor	1	Measures temperature and humidity
5	OLED Display	1	Displays real-time parameters
6	Breadboard	1	Circuit assembly
7	Jumper Wires	As required	Electrical connections

6. Pin Connections

Component	Pin	ESP32 Pin
ACS712	OUT	GPIO 34
Voltage Sensor	OUT	GPIO 35
DHT11	DATA	GPIO 4
OLED	SDA	GPIO 21
OLED	SCL	GPIO 22
Sensors / OLED	GND	GND
DHT11 / OLED	VCC	3.3V
ACS712 / Voltage Sensor	VCC	Appropriate sensor supply

Power Supply

The ESP32 can be powered through USB or a regulated 5V supply. The DHT11 and OLED display are driven from the 3.3V rail, while the ACS712 and voltage sensor module should be supplied according to their rated input voltage. All grounds are tied to a common ESP32 GND.

7. Circuit Design

The circuit consists of four major sections:

7.1 ESP32 Microcontroller

The ESP32 is the main controller. It reads the voltage, current, temperature and humidity sensor data, calculates the electrical power, drives the OLED display, and connects to Wi-Fi to upload data to ThingSpeak.

7.2 Voltage Sensor and ACS712

The voltage sensor and ACS712 current sensor are connected to the ESP32's analog input pins to measure the DG's electrical output.

Voltage Sensor OUT → ESP32 GPIO 35	**ACS712 OUT → ESP32 GPIO 34**

7.3 DHT11 Sensor

The DHT11 monitors the surrounding temperature and humidity around the DG set.

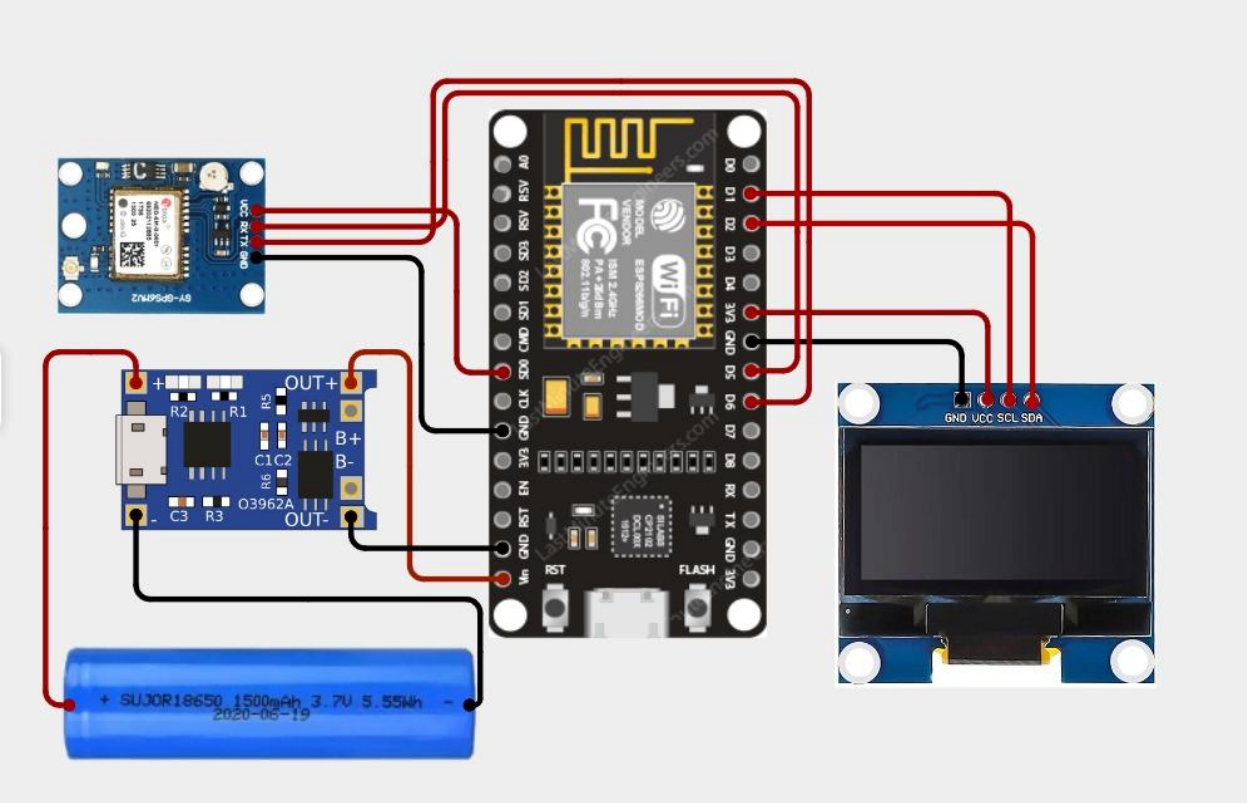
DHT11 DATA → ESP32 GPIO 4

7.4 OLED Display

The OLED display communicates with the ESP32 over I2C and shows the real-time voltage, current, power, temperature and humidity readings.

OLED SDA → ESP32 GPIO 21	**OLED SCL → ESP32 GPIO 22**

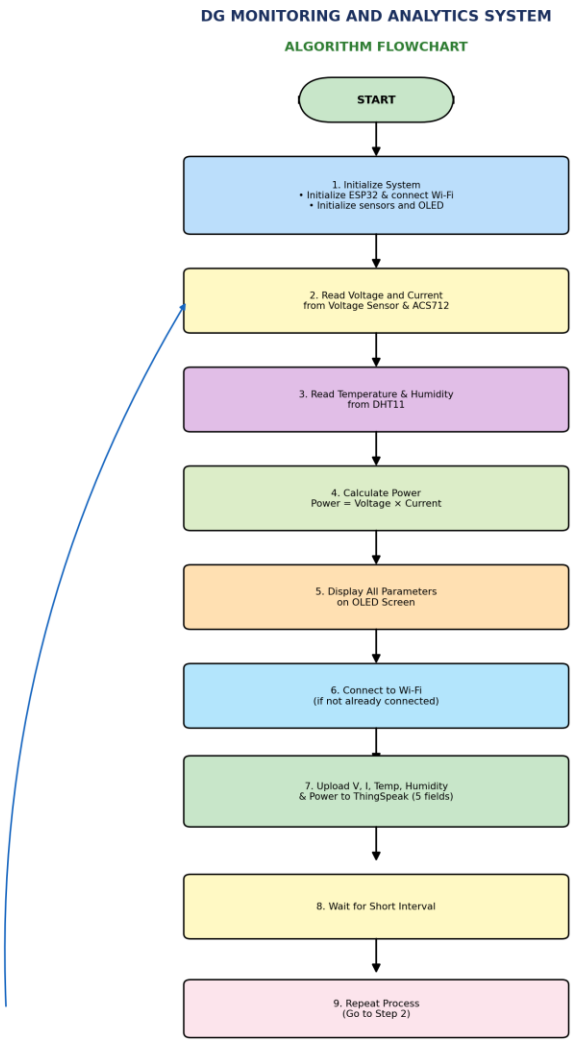
CIRCUIT DESIGN:



8. Algorithm

Algorithm

1. Start the system.
2. Initialize the ESP32, sensors and OLED display, and connect to Wi-Fi.
3. Read voltage and current from the voltage sensor and ACS712.
4. Read temperature and humidity from the DHT11.
5. Calculate power using Power = Voltage × Current.
6. Display voltage, current, power, temperature and humidity on the OLED screen.
7. Connect to Wi-Fi if not already connected.
8. Upload voltage, current, temperature, humidity and power to ThingSpeak (5 fields).
9. Wait for a short interval.
10. Repeat the process continuously.



9. Coding

The following Arduino program is designed for the ESP32 used in the project.

```
// ============================================================

// INDUSTRIAL DG MONITORING AND ANALYTICS SYSTEM

// ESP32 + ACS712 + Voltage Sensor + DHT11 + OLED

// + ThingSpeak Cloud

// ============================================================


#include <WiFi.h>
```

```
#include <HTTPClient.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>

// ================================================
// WIFI
// ================================================

const char* WIFI_SSID = "kowsi";
const char* WIFI_PASSWORD = "kowsi @ 01";

// ================================================
// THINGSPEAK
// ================================================

const char* THINGSPEAK_API_KEY =
    "52SYSQ1DR8W0JC8D";

const char* THINGSPEAK_SERVER =
    "http://api.thingspeak.com/update";

// ================================================
// OLED
// ================================================

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
```

```
#define OLED_SDA 21
#define OLED_SCL 22
#define OLED_ADDRESS 0x3C

Adafruit_SSD1306 display(
    SCREEN_WIDTH,
    SCREEN_HEIGHT,
    &Wire,
    -1
);

// ================================================================
// DHT11
// ================================================================

#define DHT_PIN 4
#define DHT_TYPE DHT11

DHT dht(DHT_PIN, DHT_TYPE);

// ================================================================
// ACS712
// ================================================================

#define CURRENT_PIN 34

// ACS712-20A = 0.100 V/A
// ACS712-5A  = 0.185 V/A
```

```
// ACS712-30A = 0.066 V/A

#define ACS712_SENSITIVITY 0.100

// ================================================================
// VOLTAGE SENSOR
// ================================================================

#define VOLTAGE_PIN 35

#define VOLTAGE_SENSOR_RATIO 5.0

// ================================================================
// ADC
// ================================================================

#define ADC_MAX 4095.0
#define ADC_REFERENCE 3.3

// ================================================================
// VARIABLES
// ================================================================

float voltageValue = 0.0;
float currentValue = 0.0;
float temperatureValue = 0.0;
float humidityValue = 0.0;
float powerValue = 0.0;
```

```
// ================================================
// READ CURRENT
// ================================================

float readCurrent()
{
  const int samples = 300;

  float sum = 0;

  for (int i = 0; i < samples; i++)
  {
    int adcValue = analogRead(CURRENT_PIN);

    float voltage =
       (adcValue / ADC_MAX) * ADC_REFERENCE;

    sum += voltage;

    delayMicroseconds(500);
  }

  float averageVoltage =
     sum / samples;

  // Approximate ACS712 zero-current voltage
  float zeroCurrentVoltage = 2.5;

  float current =
```

```
    (averageVoltage - zeroCurrentVoltage)
    / ACS712_SENSITIVITY;

  if (current < 0)
  {
    current = -current;
  }

  if (current < 0.05)
  {
    current = 0;
  }

  return current;
}

// ================================================================
// READ VOLTAGE
// ================================================================

float readVoltage()
{
  const int samples = 50;

  long total = 0;

  for (int i = 0; i < samples; i++)
  {
    total += analogRead(VOLTAGE_PIN);
```

```
  delay(2);
 }

 float averageADC =
    total / (float)samples;

 float sensorVoltage =
    (averageADC / ADC_MAX)
    * ADC_REFERENCE;

 float inputVoltage =
    sensorVoltage
    * VOLTAGE_SENSOR_RATIO;

 return inputVoltage;
}

// ================================================
// OLED DISPLAY
// ================================================

void displayData()
{
 display.clearDisplay();

 display.setTextSize(1);
 display.setTextColor(SSD1306_WHITE);
```

```
display.setCursor(0, 0);
display.println("INDUSTRIAL DG MONITOR");

display.setCursor(0, 12);
display.print("V: ");
display.print(voltageValue, 1);
display.println(" V");

display.setCursor(0, 23);
display.print("I: ");
display.print(currentValue, 2);
display.println(" A");

display.setCursor(0, 34);
display.print("P: ");
display.print(powerValue, 1);
display.println(" W");

display.setCursor(0, 45);
display.print("T:");
display.print(temperatureValue, 1);
display.print("C ");

display.print("H:");
display.print(humidityValue, 0);
display.print("%");

display.setCursor(0, 56);
```

```
  if (WiFi.status() == WL_CONNECTED)
  {
    display.print("WiFi: ONLINE");
  }
  else
  {
    display.print("WiFi: OFFLINE");
  }

  display.display();
}

// ============================================================
// CONNECT WIFI
// ============================================================

void connectWiFi()
{
  Serial.print("Connecting to WiFi");

  WiFi.begin(
      WIFI_SSID,
      WIFI_PASSWORD
  );

  int attempts = 0;

  while (
      WiFi.status() != WL_CONNECTED &&
```

```
    attempts < 30
  )
  {
    delay(500);

    Serial.print(".");

    attempts++;
  }

  Serial.println();

  if (WiFi.status() == WL_CONNECTED)
  {
    Serial.println("WiFi connected!");

    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
  }
  else
  {
    Serial.println("WiFi connection failed.");
  }
}

// ================================================
// SEND DATA TO THINGSPEAK
// ================================================
```

```
void sendToThingSpeak()
{
  if (WiFi.status() != WL_CONNECTED)
  {
    Serial.println(
        "WiFi not connected. Data not sent."
    );

    return;
  }

  HTTPClient http;

  String url =
      String(THINGSPEAK_SERVER) +
      "?api_key=" +
      String(THINGSPEAK_API_KEY) +
      "&field1=" +
      String(voltageValue, 2) +
      "&field2=" +
      String(currentValue, 2) +
      "&field3=" +
      String(temperatureValue, 2) +
      "&field4=" +
      String(humidityValue, 2) +
      "&field5=" +
      String(powerValue, 2);

  Serial.println("Sending data to ThingSpeak...");
```

```
http.begin(url);

int httpCode = http.GET();

if (httpCode > 0)
{
  String response =
      http.getString();

  Serial.print("ThingSpeak response: ");
  Serial.println(response);

  if (response != "0")
  {
    Serial.println(
        "Data uploaded successfully!"
    );
  }
  else
  {
    Serial.println(
        "ThingSpeak upload failed."
    );
  }
}
else
{
  Serial.print(
```

```
      "HTTP Error: "
    );

    Serial.println(httpCode);
  }

  http.end();
}

// ================================================================
// SETUP
// ================================================================

void setup()
{
  Serial.begin(115200);

  delay(1000);

  // ---------------- OLED ----------------

  Wire.begin(
     OLED_SDA,
     OLED_SCL
  );

  if (!display.begin(
       SSD1306_SWITCHCAPVCC,
       OLED_ADDRESS
```

```
    ))
{
  Serial.println(
      "OLED initialization failed!"
  );

  while (1)
  {
    delay(1000);
  }
}

display.clearDisplay();

display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);

display.setCursor(15, 20);
display.println("INDUSTRIAL DG");

display.setCursor(20, 35);
display.println("MONITORING");

display.display();

delay(2000);

// ---------------- DHT ----------------
```

```
  dht.begin();

  // ---------------- ADC ----------------

  analogReadResolution(12);

  analogSetPinAttenuation(
     CURRENT_PIN,
     ADC_11db
  );

  analogSetPinAttenuation(
     VOLTAGE_PIN,
     ADC_11db
  );

  // ---------------- WIFI ----------------

  connectWiFi();

  delay(1000);
}

// ================================================
// LOOP
// ================================================

void loop()
{
```

```
Serial.println();
Serial.println(
   "Reading DG parameters..."
);

// ---------------- CURRENT ----------------

currentValue =
   readCurrent();

// ---------------- VOLTAGE ----------------

voltageValue =
   readVoltage();

// ---------------- DHT11 ----------------

float newHumidity =
   dht.readHumidity();

float newTemperature =
   dht.readTemperature();

if (!isnan(newHumidity) &&
   !isnan(newTemperature))
{
  humidityValue =
     newHumidity;
```

```
  temperatureValue =
      newTemperature;
}
else
{
  Serial.println(
      "DHT11 reading failed!"
  );
}

// ---------------- POWER ----------------

powerValue =
    voltageValue *
    currentValue;

// ---------------- SERIAL ----------------

Serial.println(
    "----------------------------"
);

Serial.print("Voltage: ");
Serial.print(voltageValue, 2);
Serial.println(" V");

Serial.print("Current: ");
Serial.print(currentValue, 2);
Serial.println(" A");
```

```
  Serial.print("Power: ");
  Serial.print(powerValue, 2);
  Serial.println(" W");

  Serial.print("Temperature: ");
  Serial.print(temperatureValue, 1);
  Serial.println(" C");

  Serial.print("Humidity: ");
  Serial.print(humidityValue, 1);
  Serial.println(" %");

  // ---------------- OLED ----------------

  displayData();

  // ---------------- THINGSPEAK ----------------

  sendToThingSpeak();

  // ThingSpeak requires at least
  // about 15 seconds between updates
  delay(20000);
}
```

Software Used

- Arduino IDE
- ESP32 Board Package

- DHT Sensor Library
- Adafruit GFX Library
- Adafruit SSD1306 Library
- ThingSpeak Cloud Platform

Code Explanation

- GPIO 35 reads the voltage sensor; GPIO 34 reads the ACS712 current sensor.
- GPIO 4 receives data from the DHT11.
- GPIO 21/22 (I2C) drive the OLED display through the Adafruit_SSD1306 library.
- power is calculated in software as voltage × current.
- voltageCalibration and currentSensitivity are calibration constants that should be adjusted for the actual sensors used.
- ThingSpeak.setField() and ThingSpeak.writeFields() upload all five parameters (voltage, current, temperature, humidity, power) to the ThingSpeak channel using the Write API Key.

10. ThingSpeak Configuration

The ThingSpeak channel was configured with five fields:

Field	Parameter
Field 1	Voltage
Field 2	Current
Field 3	Temperature
Field 4	Humidity
Field 5	Power

The ESP32 sends the sensor data to ThingSpeak through the ThingSpeak Write API Key using Wi-Fi.

11. Bill of Materials

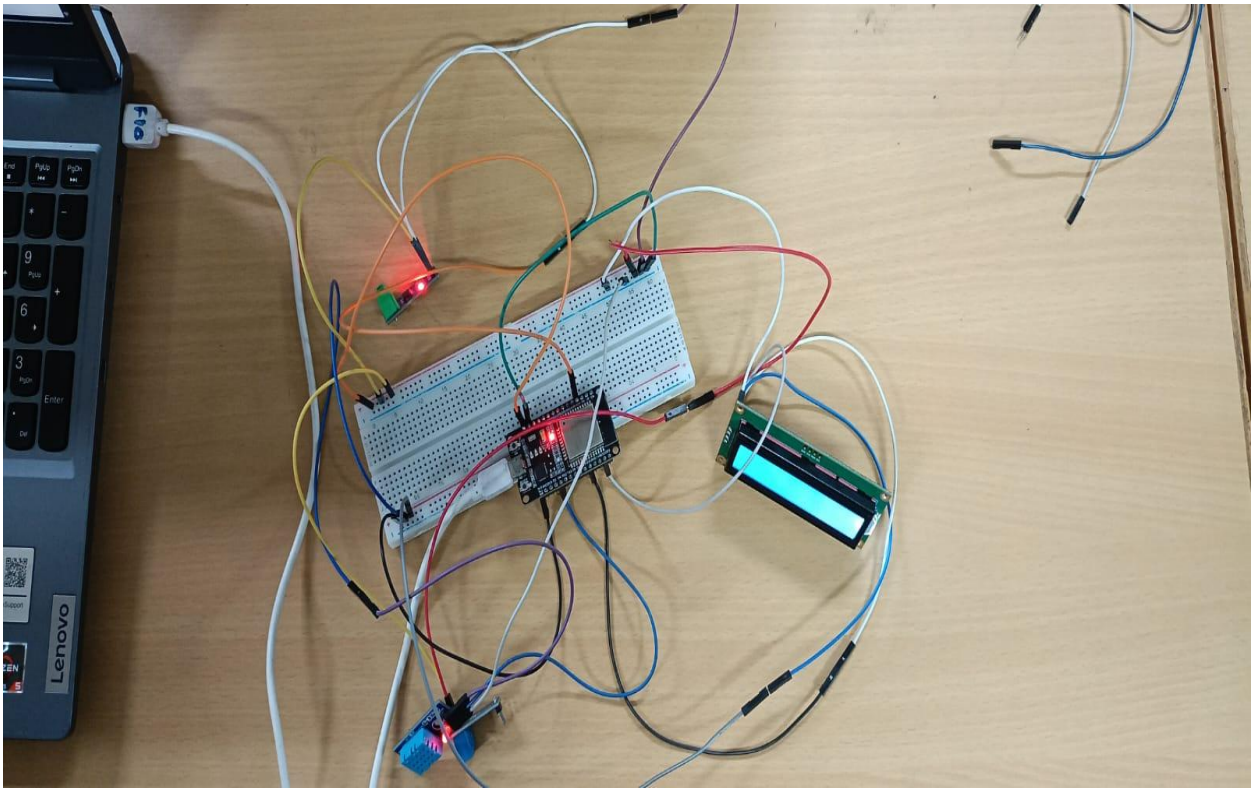
S.No.	Component	Quantity
1	ESP32 MCU	1
2	ACS712 Current Sensor	1
3	Voltage Sensor Module	1
4	DHT11 Sensor	1
5	OLED Display	1
6	Breadboard	1
7	Jumper Wires	As required

12. Prototype Implementation

The prototype was assembled using an ESP32 development board, a voltage sensor module, an ACS712 current sensor, a DHT11 sensor, and an OLED display, wired on a breadboard as per the pin connections above.

The ESP32 was selected as the main controller because it provides sufficient analog input channels for the voltage and current sensors, along with built-in Wi-Fi connectivity for direct integration with ThingSpeak, without requiring an additional communication module.

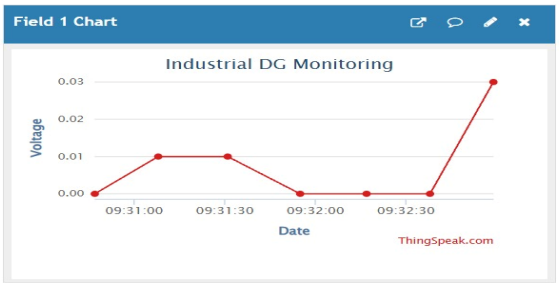
The prototype continuously reads voltage, current, temperature and humidity, calculates the approximate power, displays all parameters on the OLED, and streams the readings to the ThingSpeak channel.

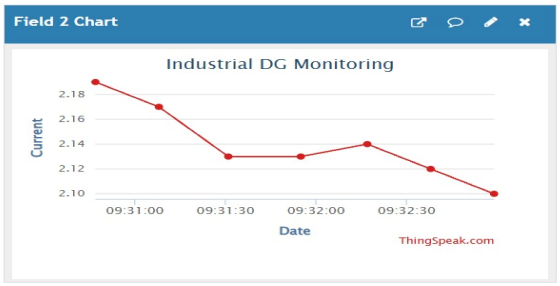


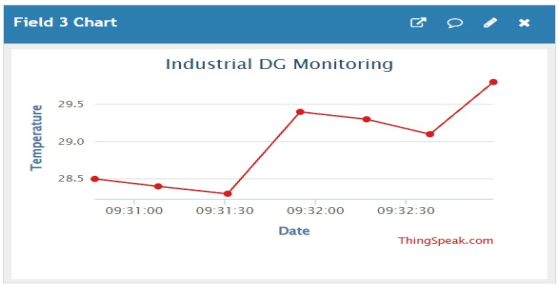
13. Expected Result

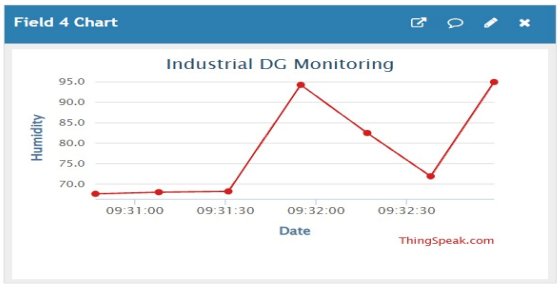
The developed system successfully measures voltage, current, temperature, humidity, and calculated power. The values are displayed on the OLED and uploaded to ThingSpeak through Wi-Fi. The ThingSpeak dashboard provides graphical visualization of the DG operating parameters, enabling convenient remote monitoring and basic performance analysis.

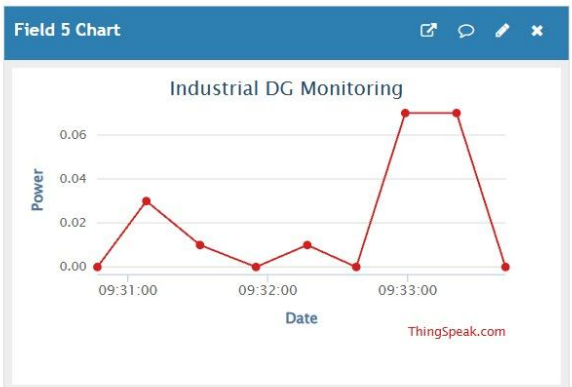
Parameter	Local Display	Cloud Field
Generator Voltage (V)	OLED	Field 1
Generator Current (A)	OLED	Field 2
Temperature (°C)	OLED	Field 3
Humidity (%)	OLED	Field 4
Power (W)	OLED	Field 5











14. Advantages

- Real-time DG monitoring.
- Wireless data transmission.
- Remote monitoring through ThingSpeak.
- Low-cost prototype.
- Compact ESP32-based system.
- Multiple parameters can be monitored simultaneously.
- Historical data can be viewed through cloud graphs.

15. Applications

- Industrial DG sets
- Backup power systems
- Manufacturing industries
- Commercial buildings
- Hospitals
- Data centers
- Industrial energy monitoring systems

16. Conclusion

The Industrial DG Monitoring and Analytics System provides an IoT-based approach for monitoring important generator parameters. By integrating the ESP32, ACS712, voltage sensor, DHT11, OLED display, and ThingSpeak, the system provides both local and remote monitoring.

This can support better observation of DG operating conditions and provide useful historical data for analysis and maintenance planning. In future development, additional features such as RPM sensing, fuel-level monitoring, mobile-app notifications, and machine-learning-based fault prediction can be incorporated to further extend the system's analytics capabilities.